Research Paper Poster Presentation & Review Report

Course Code: SWE 4701
Course Name: Software Metrics and Process

Report By:

Namisa Najah Raisa
210042112

How do Machine Learning Projects use Continuous Integration Practices? An Empirical Study on GitHub Actions

João Helis Bernardo, Sérgio Queiroz de Medeiros, Uirá Kulesza [Federal University of Rio Grande do Norte]

Daniel Alencar da Costa [University of Otago]

Published in: Mining Software Repositories 2024 conference

Date: March 12, 2026

1 Summary of the Paper

Machine learning (ML) systems are increasingly integrated into modern software, yet the engineering practices supporting them remain underexplored. While Continuous Integration (CI) is well established in traditional software development, its use and effectiveness in ML projects are less understood.

The paper addresses this gap through an empirical study of **185 GitHub projects** using GitHub Actions: 93 ML and 92 non-ML repositories. It examines three questions: **(RQ1)** differences in CI practices between ML and non-ML projects, **(RQ2)** how CI metrics evolve over time, and **(RQ3)** the CI-related concerns developers raise in pull request discussions.

Results show notable differences. ML projects have significantly longer build durations: small ML projects average **10.3 minutes** compared to **0.81 minutes** in non-ML projects, while medium projects require **13 minutes** versus **5.54 minutes**. Medium ML projects also have lower test coverage (**83%** vs **94%**). Build durations tend to increase over time, and PR discussions reveal a wider range of CI concerns, particularly failures unrelated to code changes.

Overall, the study finds that CI in ML projects is more complex and tends to deteriorate over time, indicating the need for CI practices tailored to ML development.

2 Critical Evaluation of Methodology

2.1 Project Selection and Dataset Construction

The study builds on a curated dataset from Gonzalez et al. (2020), later refined by Rzig et al. (2022). A multi-stage filtering process selects repositories with active GitHub Actions workflows, at least 100 workflow runs, and a minimum of six months of CI history, ensuring that only projects with sustained CI usage are included. To maintain balance between project types, the authors supplemented the non-ML group with highly starred GitHub repositories, resulting in a dataset of 93 ML and 92 non-ML projects.

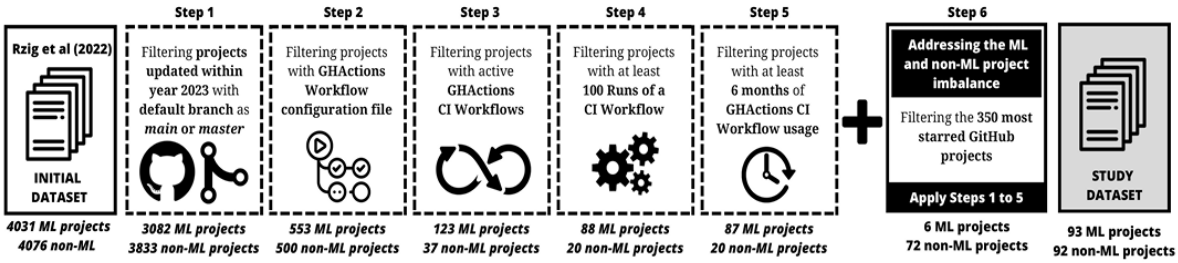


Figure 1: An overview of the project selection process.

However, using highly starred repositories may introduce bias, as these projects often receive greater maintenance effort and may perform better on quality metrics. Additionally, project size is measured using Lines of Code (LOC), which reflects code volume but does not capture the computational demands typical of ML workloads.

2.2 CI Workflow Classification

Workflows are classified as CI or non-CI through keyword matching in job and step names, validated against a manual sample of 100 workflows yielding 100% precision and 86.7% recall. The precision result is the most consequential: no non-CI workflow was mislabeled, meaning the analysis is free from contamination by irrelevant pipeline data. The 13% recall gap implies some genuine CI workflows were excluded, which could introduce selection bias if ML projects are disproportionately affected.

2.3 Quantitative and Temporal Analysis

For RQ1, the Mann–Whitney–Wilcoxon test with Cliff's delta is used to analyse differences in non-normally distributed data. Reporting effect sizes alongside p-values helps distinguish statistically significant results from practically meaningful ones.

For RQ2, Dynamic Time Warping (DTW) clustering with gap statistics compares build duration trends across projects with time series of varying lengths. However, trend labels (increasing, decreasing, or stable) are assigned through visual inspection of cluster centroids rather than formal statistical tests, introducing some subjectivity.

2.4 Qualitative Analysis

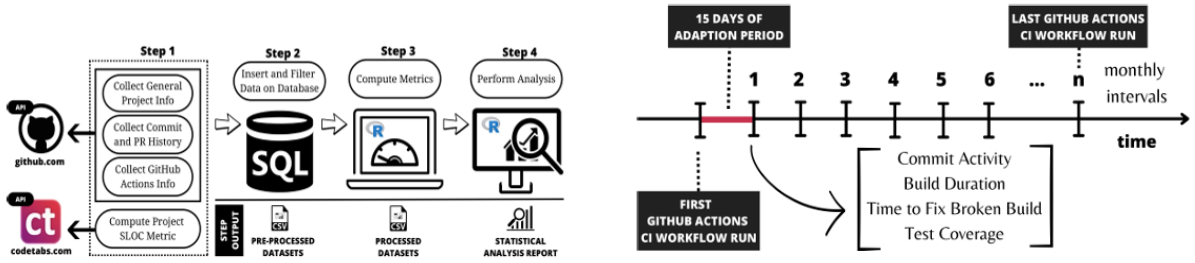
The qualitative study analyses 719 stratified pull requests using inductive thematic analysis. Multiple researchers independently coded the PR discussions and later reconciled their interpretations, reducing individual bias. Although the authors do not report inter-rater reliability metrics, they consider collaborative reconciliation sufficient. The resulting themes are coherent and align with the quantitative findings.

3 Detailed Analysis of Results

3.1 Metrics and Measurement Approach

Four metrics are used to evaluate CI practice adoption. Commit Activity measures the proportion of days with at least one commit within a monthly interval. Build Duration is the median runtime of successful CI runs, excluding failed runs that terminate prematurely. Time to Fix Broken Builds records the median number of hours a failed workflow remains unresolved. Test Coverage, obtained from Coveralls and CodeCov, represents the percentage of code exercised by automated tests.

Metrics are calculated over 30-day intervals following a 15-day stabilization period after CI adoption, and project-level medians are used for comparisons.



(a) Data collection process

(b) Periods of analysis

Figure 2: Overview of the data collection and analysis period

3.2 RQ1: Differences in CI Practice Adoption

Build duration represents the most significant difference between ML and non-ML projects. Small ML projects have build times roughly **12 times longer** than small non-ML projects, and the gap remains substantial for medium-sized projects. However, for large projects the difference is not statistically significant. Although Python's dynamic typing was initially considered a possible explanation, language-controlled comparisons show that the disparity persists across both static and dynamic languages. This suggests that ML-specific tasks such as data preprocessing, model validation, and computationally intensive tests contribute to longer CI pipelines.

Test coverage shows a significant difference only for medium-sized projects, where ML repositories achieve **83%** coverage compared to **94%** in non-ML repositories. No significant differences are observed in commit activity or the time required to fix broken builds, indicating similar development pace and recovery behaviour across both project types.

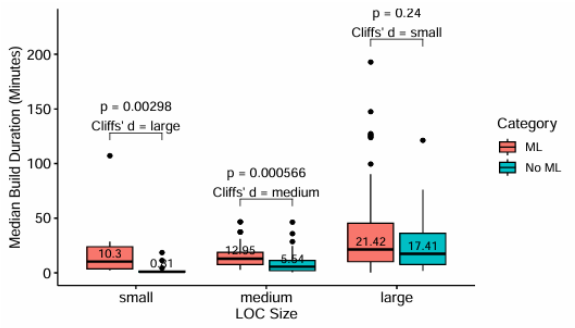


Figure 3: Build duration by project category and size.

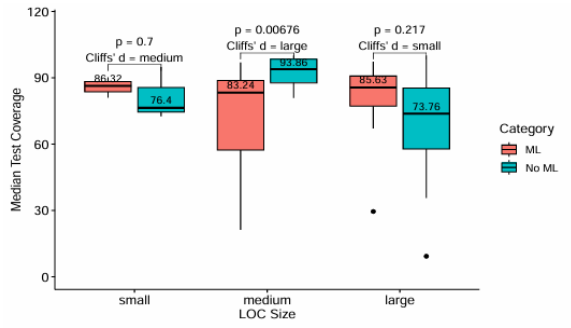
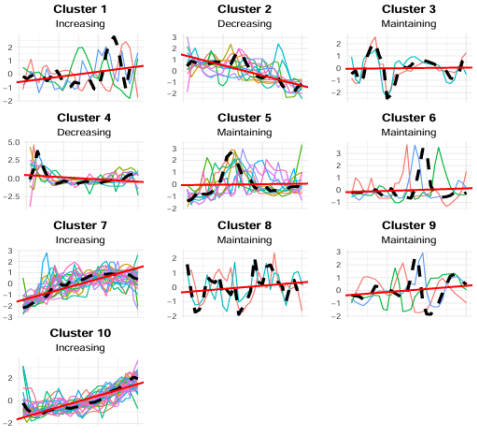


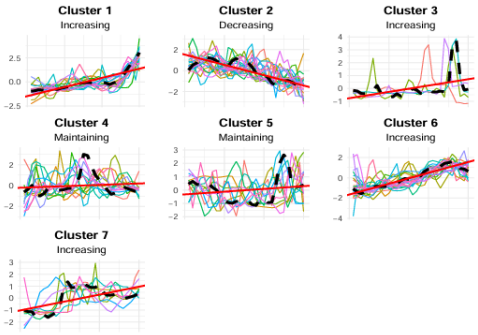
Figure 4: Test coverage by project category and size.

3.3 RQ2: Temporal Evolution of CI Metrics

DTW clustering identifies 10 build duration trend clusters in ML projects and 7 in non-ML projects, indicating greater trajectory diversity in ML repositories.



(a) Build duration clustering patterns in ML projects



(b) Build duration clustering patterns in non-ML projects

Figure 5: Comparison of build duration clustering trends in ML and non-ML projects

Increasing build duration trends are common in ML projects. Among small repositories, **75%** of ML projects show increasing trends compared to **35.7%** of non-ML projects, while for medium projects the figures are **61.4%** and **44.7%**. This suggests that the build duration gap identified in RQ1 is worsening over time.

For large projects, the pattern reverses: non-ML projects show more increasing trends (65%) than ML projects (51.2%), possibly because larger ML repositories have implemented optimizations such as workflow parallelization or caching.

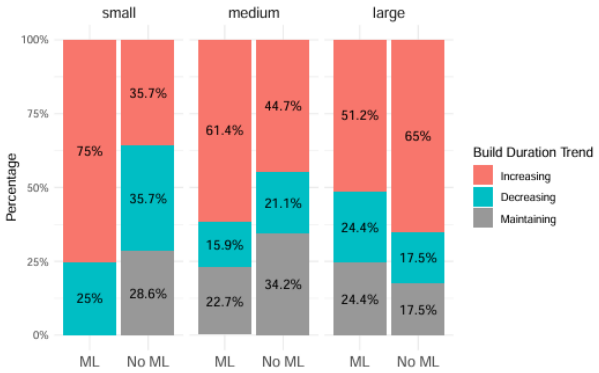


Figure 6: Clustering trends per project category and size.

Test coverage trends are mostly stable for both groups. However, several ML projects maintain relatively low coverage, with nearly half of medium repositories remaining below 75%.

3.4 RQ3: CI Discussions in Pull Requests

ML projects exhibit a broader range of CI-related discussions, with **73 themes** compared to **23** in non-ML repositories.

Both groups discuss common topics such as build status, debugging, configuration, and test additions. However, ML projects more frequently address the *relatedness of failures*, where CI failures occur independently of the submitted code changes. This issue, often caused by ML non-determinism and stochastic training processes, contributes to a theme of *mistrust in CI* unique to ML projects, where developers question the reliability of CI results.

3.5 Sufficiency of Evidence and Validation

The quantitative conclusions are well-substantiated by the statistical framework employed, and internally consistent across RQ1 and RQ2. The qualitative results are coherent and gain credibility from their alignment with the quantitative picture. The principal analytical gap is the absence of multivariate modeling: examining each metric in pairwise isolation, without accounting for the combined influence of project age, contributor count, or language type, leaves open the question of how much of the observed difference is genuinely attributable to ML-specific characteristics versus correlated project-level variables.

4 Threats to Validity and Future Research Directions

The authors acknowledge several validity threats. For internal validity, reliance on third-party tools such as **CodeTabs**, **Coveralls**, and **CodeCov** introduces limited transparency in metric collection. The age difference between **ML (5.7 years)** and **non-ML (8.4 years)** projects is also noted but not controlled for, meaning older projects may simply have more mature CI practices. Regarding external validity, the study only examines public GitHub repositories using GitHub Actions, which limits generalization to private repositories and other CI systems like Jenkins or GitLab CI.

These limitations suggest several directions for future research. In particular, the causes of false positives in ML CI pipelines require deeper investigation, including factors such as stochastic training, unstable data pipelines, or environment inconsistencies. Applying causal inference methods and incorporating developer interviews could also provide clearer insights beyond what repository data alone reveals.

5 Personal Reflections

This paper prompted a reassessment of how Continuous Integration (CI) fits within machine learning engineering. CI is often treated as a background operational tool compared to model performance or data quality. However, the study highlights that the long-term reliability of ML systems depends not only on model accuracy but also on the engineering infrastructure that manages integration, testing, and validation.

One of the most notable findings concerns build duration. While Python's dynamic typing might appear to be the cause, the real factors are structural: heavy data preprocessing, numerous model configurations requiring validation, and the computational cost of ML tests. This suggests that architectural strategies such as workflow caching and selective test execution may be more effective than language-level optimizations.

Another important insight is the issue of CI mistrust. Unlike traditional software development, where failing builds clearly indicate problems, ML builds can fail due to randomness or unstable data dependencies. As a result, developers may begin to question CI signals. Improving reproducibility, test isolation, and CI tools designed for ML's inherent non-determinism will be crucial for maintaining trust in ML development workflows.